



**CSE3433 SOFTWARE ARCHITECTURE
K2**

Sem II 2025/2026

**Program Sarjana Muda Sains Komputer (Kejuruteraan
perisian) Dengan Kepujian**

GROUP 15

Project Proposal

NAMA PENSYARAH: DR MOHAMAD NOR

	NAMA	NO MATIRK
1	MUHAMMAD RADILAN BIN YAAKUB	S75736
2	MUHAMMAD ADAM DANISH BIN MOHAMAD FAIZ	S75851

Table of Contents

1. Architecture Design.....	2
1.1 System Overview.....	2
1.2 Functional Requirements.....	2
1.3 Non-functional Requirements.....	3
1.4 Architecture Style Explanation.....	3
1.5 Service or Component Identification.....	4
1.6 Architecture Diagrams.....	5
1.6.1 System Context Diagram.....	5
1.6.2 Component / Service Architecture Diagram.....	6
1.6.3 Interaction Diagram.....	7
1.6.4 Deployment Diagram.....	8
2. Technology Stack.....	9
3. Prototype Implementation.....	11
4. Architecture Evaluation.....	15
4.1 Elastic Scalability.....	15
4.2 High Availability.....	15
4.3 Bulletproof Persistence.....	16
4.4 Responsiveness.....	16
5. Team Contributions.....	17
6. Conclusion and Reflection.....	18
6.1 Conclusion.....	18
6.2 Reflection.....	19
References.....	20

1. Architecture Design

1.1 System Overview

Event-Driven Job Recruitment & Application Processing System is the system prototype that have been create for modernize and automated the recruitment process with more efficiently and systematically. This system allows two types of user which is employers and job seeker have interaction in one platform. Employers can advertise vacancies while job seeker can apply work online. Each interaction in this system such as submitting applications, updating status, and scheduling interviews will trigger certain events that will be processed automatically by other components.

Furthermore, this system operates asynchronously and does not depend directly on each other. Therefore, it is also increasing the flexibility and scalability of the system. For example, when a user submits a job application, the system will generate an “Application Submitted” event which will then trigger other processes such as data storage, initial assessment, and sending notifications to users. This approach will allow the system to provide real-time responses and reduce delays in data processing. As a result, this system aims to provide a more responsive, modular, and suitable solution for the needs of modern systems in recruitment management.

1.2 Functional Requirements

To get the job done, the system focuses on a few non-negotiable features:

- **Job Lifecycle Management:** Hiring managers need to be able to spin up new job requirements and have those requirements automatically blast out to internal and external boards via event triggers.
- **Smooth Application Flow:** Candidates need a clean portal where they can drop their details and resumes. The system must acknowledge receipt instantly, even if the backend is still "thinking".
- **Intelligence on the Fly:** We need a background service that automatically rips data out of PDFs (parsing) and matches candidate skills against the job's "must-haves" (scoring).
- **Real-time Status Loop:** Every time a candidate moves from "Applied" to "Shortlisted" or "Rejected," the system needs to fire off a notification so no one is left in the dark.

1.3 Non-functional Requirements

This is where the "engineering" happens to ensure the system doesn't break under pressure:

- **Elastic Scalability:** If a viral job post brings in 10,000 applications in an hour, the message broker (like Kafka or RabbitMQ) acts as a buffer so our database doesn't catch fire.
- **High Availability:** The candidate portal should almost never be "down". By decoupling the services, even if the "Resume Scorer" is undergoing an update, candidates can still submit their applications.
- **Bulletproof Persistence:** No application should ever be "lost". Using an event-driven approach means if a service crashes, the event stays in the queue until it's successfully processed.
- **Responsiveness:** The goal is a sub-2-second response time for any front-end action, offloading all the "heavy" work to background consumers.

1.4 Architecture Style Explanation

We chose **Event-Driven Architecture (EDA)** because, frankly, the old way of doing things is too rigid for modern hiring. In a traditional setup, if the email server is slow, the whole application process feels slow for the candidate. In this architecture, every actions or change in this system will create one event that will be sent to another komponen through intermediary which is event broker. The components in the system do not communicate directly, but instead respond to the events received. It is also makes the system loosely coupled and more flexible to develop and expand. This approach is very suitable for recruitment systems that involve various processes such as submitting applications, evaluating candidates, and sending notifications that occur continuously.

Furthermore, EDA support asynchronous processing where every event can be process without waiting for immediate response from another component. Therefore, the system can operate more efficiently and reduce the risk of delay especially when numbers of user increase. For example, when event like "Application Submitted" is generate, the system will simultaneously save the data of application and then the system will be send notification to the user. After that, the system will start the evaluation process without disturbing main flow of the system. For the conclusion, the use of Event-Driven Architecture not only improves the performance and scalability of the system, but it is also ensures that the system is easier to maintain and suitable for the needs of modern applications.

1.5 Service or Component Identification

1. API Gateway

Acts as the main entry gate for all user request which will route the request to the appropriate service as well as manage authentication and communication control.

2. Event Bus (Kafka or RabbitMQ)

Function as a communication layer which enable interaction between service asynchronous through publishing concept and event subscription.

3. User Management Service

Manage all function related to user including application, log in, identity verification, and profile management for job seeker and employers.

4. Job Service

Handle all operations related to the job vacancies such as creating, updating, deleting, and displaying job postings in the system.

5. Interview Scheduling Service

Responsible to manage interview scheduling including date and time, and also coordinating between employers and candidates.

6. Application Processing Service

The engine for handling incoming applications, resume storage, and the state-machine logic for candidate progression.

7. Notification Service

Send notifications to the user in real time based on event in the system such as update application status and interview schedule.

1.6 Architecture Diagrams

1.6.1 System Context Diagram

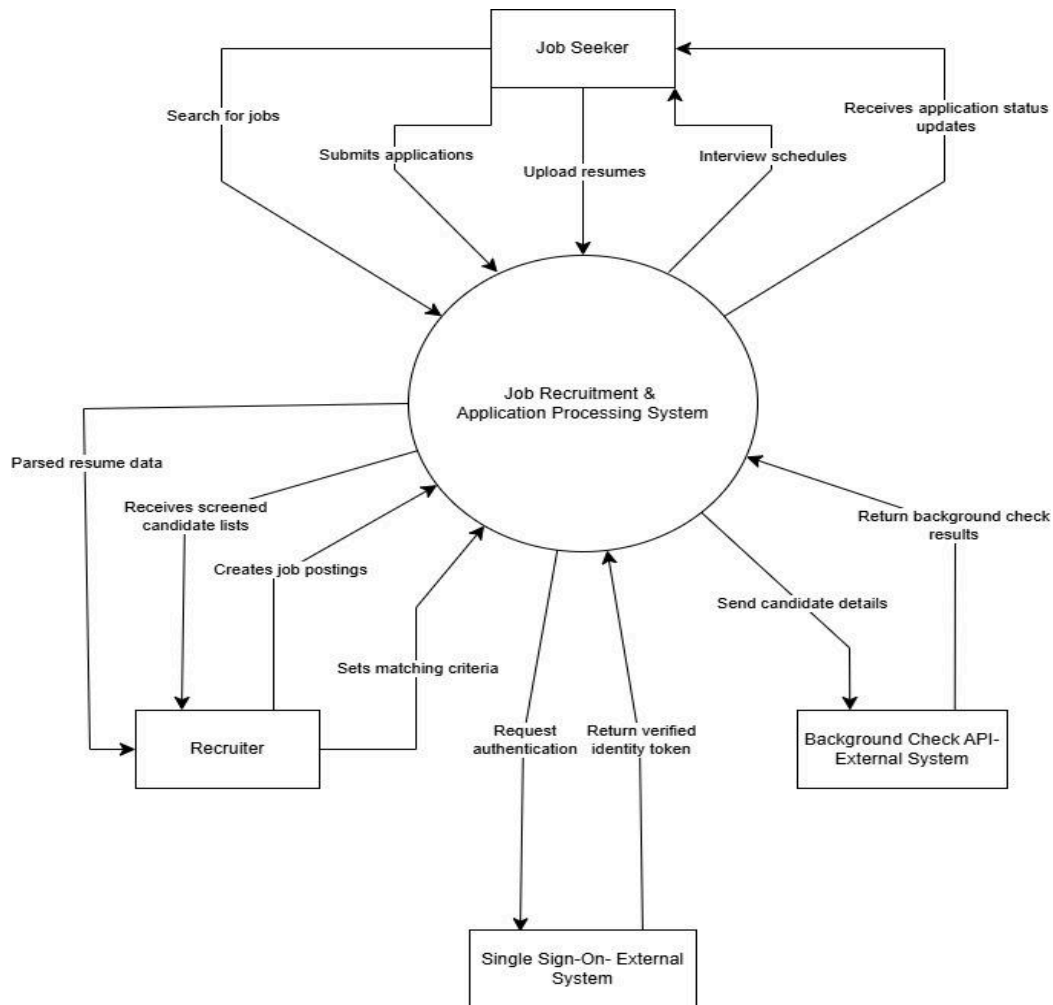


Diagram 1 : System Context Diagram

The system illustrates a **Job Recruitment & Application Processing System** as a main central which is connects multiple user and external system. **Job seeker** interacts with the the system for searching a job, submitting applications, and uploading resumes . It is also can receive interview schedule and update application status. On other side, **recruiter** use the system to create job postings, sets candidate criteria and also receive list of candidates which is have been screened along with the processed resume data. This system also integrate with the **Single Sign-On External** for the identity verification where system will send an authentication request and receive a verified identity token to ensure access security. In addition, the system also communicates with the **Background Check API External** system by sending candidate information and receiving back the results of the check.

1.6.2 Component / Service Architecture Diagram

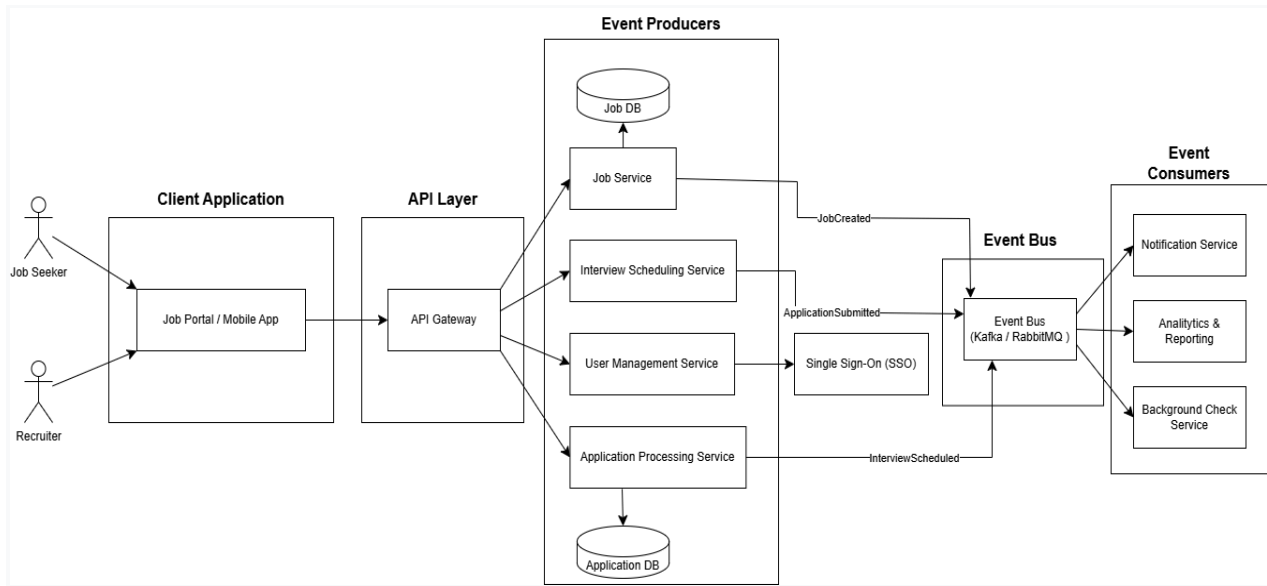


Diagram 2 : Component/Service Architecture Diagram

The Component / Service Architecture diagram illustrates an event-driven architecture for the Job Recruitment and Application Processing System. User such as Job Seeker And Recruiter interact through the client application which is Job Portal or Mobile App. It will be send a request to the API Gateway in the API Layer. After that, gateway distributes requests to several services such as Job Service (managing job data), Interview Scheduling Service (scheduling interviews), User Management Service (user management and SSO), and Application Processing Service (processing applications). Each service can store data in its own database and also produce events such as “JobCreated”, “ApplicationSubmitted”, and “InterviewScheduled” which are sent to the Event Bus (Kafka or RabbitMQ). The Event Bus then delivers these events to event consumers such as Notification Service, Analytics & Reporting, and Background Check Service for further action asynchronously, making the system more flexible, scalable, and loosely coupled.

1.6.3 Interaction Diagram

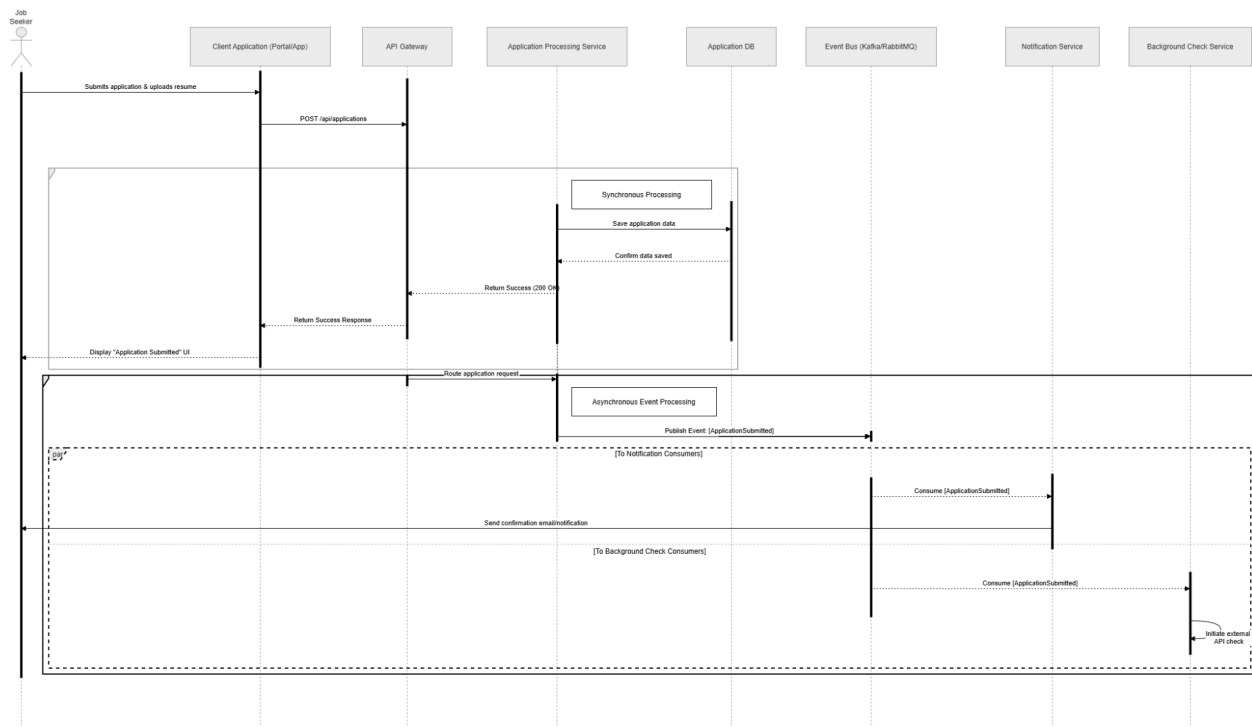


Diagram 3 : Interaction Diagram

The steps involved in a job seeker submitting an application are depicted in the interaction diagram. The process begins with the **Client Application** sending the application data to the **API Gateway**, which then forwards the request to the **Application Processing Service**. An **Application Submitted** event is instantly published to the Event Bus via this service. After that, the Event Bus initiates two simultaneous processes: the **Notification Service** gives the user an immediate confirmation, and the **Application Processing Service**, operating as a background consumer, does a preliminary assessment. Following evaluation, the consumer publishes an **ApplicationReviewed** event, which causes the **Notification Service** to provide a final status update. This flow illustrates the Event-Driven Architecture's asynchronous and decoupled design, which guarantees the system's responsiveness even when processing demanding activities.

1.6.4 Deployment Diagram

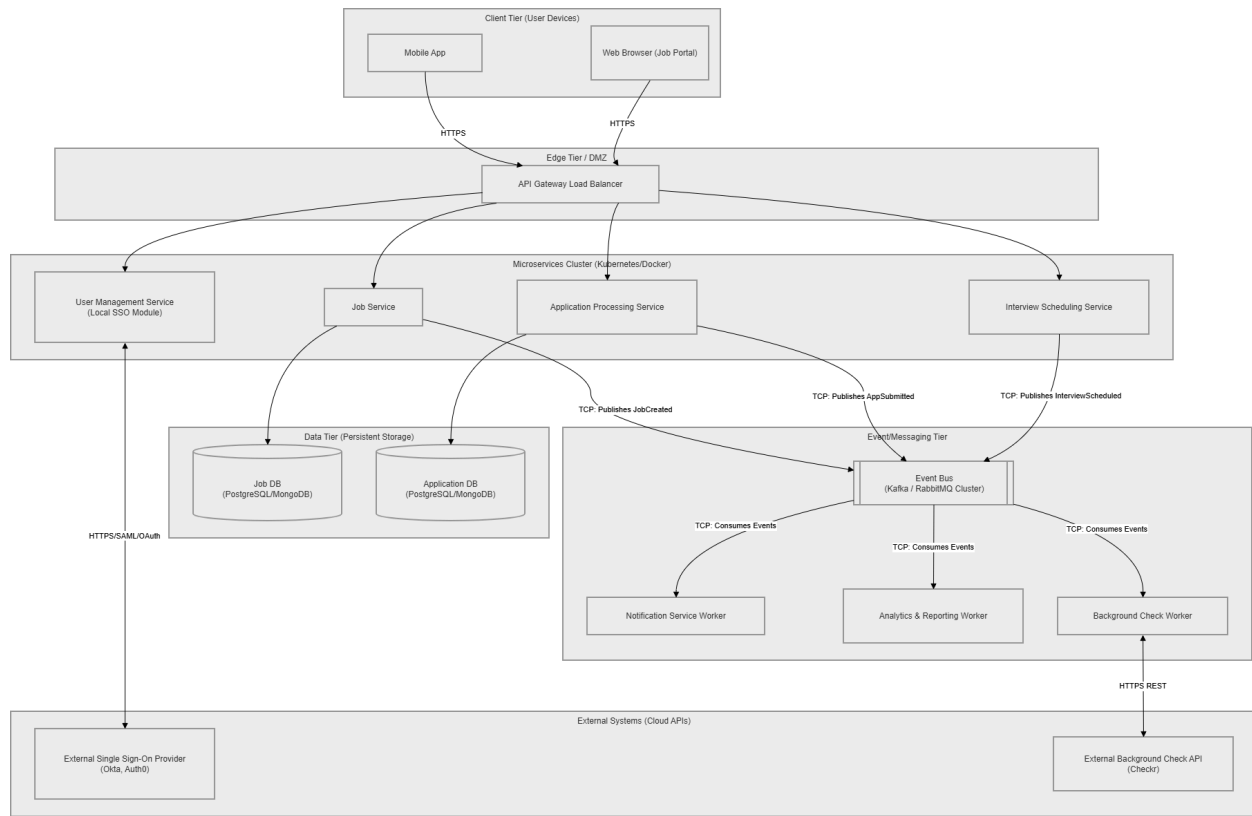


Diagram 4 : Deployment Diagram

The Event-Driven Job Requirement & Application Processing System's deployment environment and physical infrastructure are depicted in the deployment diagram. Request routing and traffic management are handled by an API gateway, which controls client access through web or mobile applications. A collection of decoupled **Microservices** like User Service, Job Service, and Application Processing Service that are deployed in containers like Docker for high availability and elastic scalability provide the system's essential functionality. The **Event Bus** (Kafka or RabbitMQ), which is necessary for controlling the asynchronous communication between all services, is hosted on a dedicated server node. The Data Layer ensures high availability and unbreakable durability by maintaining persistent data across one or more databases.

2. Technology Stack

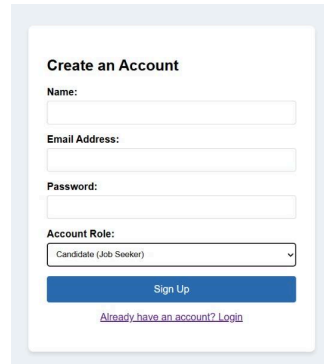
At the Frontend, standard web technologies including HTML, CSS, and JavaScript are used to build the user interface, while JavaServer Pages (JSP) is utilized to generate dynamic web content across various system views. These include the main landing page (`index.html`), authentication pages (`login.jsp`, `signup.jsp`), user workspaces (`dashboard.jsp`, `workspace.jsp`), and job management pages (`jobs.jsp`, `applyJob.jsp`, `success.jsp`). The user interactions and form submissions on these pages send HTTP requests directly to the backend controllers. The frontend assets and compiled web pages are structured inside the `build/web/` directory, managed under the standard WEB-INF and META-INF configurations for Java web archives.

While in the Backend, the system is built as a native Java Web Application running on a standard servlet container (such as Apache Tomcat or GlassFish). It processes the core application logic through the Model-View-Controller (MVC) architectural pattern. The system exposes its server-side logic through six controller classes (Servlets) inside the controller package: `LoginServlet`, `SignupServlet`, `CreateJobServlet`, `ApplyJobServlet`, `SearchServlet`, and `ScheduleInterviewServlet`. These controllers intercept incoming client requests, process user inputs, and coordinate with the business logic and data layers. Security features, specifically secure credential handling, are managed within the `util` package using `PasswordUtil.java` to perform password hashing. Specialized business workflows, such as processing application states and triggering notifications, are encapsulated within the `service` package through `ApplicationProcessor` and `NotificationService`.

For the Data Access Layer, the system strictly implements the Data Access Object (DAO) pattern to isolate the business logic from the persistence layer. This layer consists of four distinct DAO classes inside the `dao` package: `UserDAO`, `JobDAO`, `ApplicationDAO`, and `InterviewDAO`. All direct database operations such as executing SQL queries, managing result sets, and performing CRUD operations are handled exclusively by these DAO classes. Database connectivity is initialized and managed globally via the `DBConnection.java` utility class inside the `util` package, which configures the JDBC connection parameters.

For the Database Layer, a MySQL relational database is used to persist all system records. The complete relational schema, including tables for users, jobs, applications, and interview schedules, is defined and initialized via the `jobportal_db.sql` script located at the project root. For the application build and lifecycle management, the project utilizes Apache Ant as the build automation tool, configured through `build.xml` and the configuration files within the `nbproject/` directory (such as `project.xml` and `project.properties`), ensuring a consistent compilation, packaging into a WAR deployment file (found in the `dist/` folder), and execution environment during development.

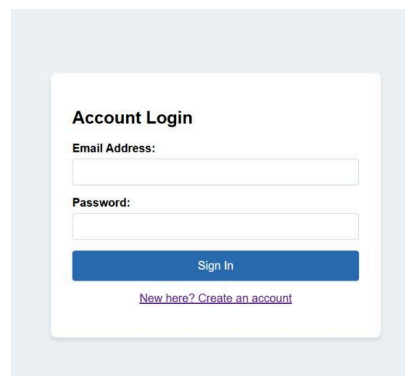
3. Prototype Implementation



The Sign Up Interface is a web form titled "Create an Account". It contains four input fields: "Name:", "Email Address:", "Password:", and "Account Role:". The "Account Role:" field is a dropdown menu with "Candidate (Job Seeker)" selected. Below the fields is a blue "Sign Up" button and a link that says "Already have an account? Login".

Diagram 3.1 : Sign Up Interface

Diagram 3.1 shows the Sign Up Interface of the Workspace. Users need to input their name, account and password before continue browsing.



The Sign In Interface is a web form titled "Account Login". It contains two input fields: "Email Address:" and "Password:". Below the fields is a blue "Sign In" button and a link that says "New here? Create an account".

Diagram 3.2 : Sign In Interface

Diagram 3.2 shows the Sign In Interface. Users can access the platform through login by entering their email address with their registered passwords.

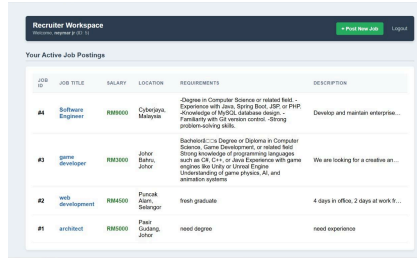


Diagram 3.3 : Recruiter Workspace Interface

Diagram 3.3 shows the Recruiter Workspace for Recruiter sign in interface. They can see the previous added jobs or add a new jobs.

Create Job Vacancy

Diagram 3.4 : Adding Job Interface

Diagram 3.4 shows Adding Job Interface that will allow Recruiter to add new job by input Job Title, Salary, Location, Job Description and Requirement.

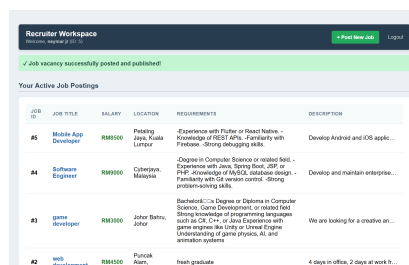


Diagram 3.5 : Successful Adding Job Interface

Diagram 3.5 shows the successful Adding Job Interface. This interface will inform the Recruiter that the new job has been created.

CANDIDATE	JOB	EDUCATION	SKILLS	EXPERIENCE	EMPLOYED	STATUS	ACTION
MU	Software Engineer	Diploma	Java, SQL, Spring, Lombok	Internship	Software Development	REJECTED	Accept Reject Interview
Adam	web development	Bachelor Degree	Java	Fresh Graduate	Software Development	INTERVIEW_SCHEDULED	Accept Reject Interview
Adam	game developer	Bachelor of Software Engineering (CSC) (2018-2022)	Spring Boot, Java, JSP, JST, PHP, MySQL, Docker	Software Engineer of Top Software Company	Backend development, API Development, System Design	INTERVIEW_SCHEDULED	Accept Reject Interview
Adam	architect	null	null	null	null	INTERVIEW_SCHEDULED	Accept Reject Interview

Diagram 3.6 : Received Job Application Interface

Diagram 3.6 show Received Job Application Interface that allows Recruiter to see candidate application and they can accept, reject or make an interview for candidates who apply the job.

Candidate Dashboard														
Welcome, Adam (11.11)														
Applications 3	Shortlisted 1	Interviews 2												
Rejected 0														
Search Jobs Search														
Application Tracking <table border="1"> <thead> <tr> <th>Job</th> <th>Date</th> <th>Status</th> </tr> </thead> <tbody> <tr> <td>web development</td> <td>2020-08-20 09:59:39.0</td> <td>Interview Scheduled</td> </tr> <tr> <td>game developer</td> <td>2020-06-19 22:40:11.0</td> <td>Rejected</td> </tr> <tr> <td>architect</td> <td>2020-06-19 10:14:05.0</td> <td>Interview Scheduled</td> </tr> </tbody> </table>			Job	Date	Status	web development	2020-08-20 09:59:39.0	Interview Scheduled	game developer	2020-06-19 22:40:11.0	Rejected	architect	2020-06-19 10:14:05.0	Interview Scheduled
Job	Date	Status												
web development	2020-08-20 09:59:39.0	Interview Scheduled												
game developer	2020-06-19 22:40:11.0	Rejected												
architect	2020-06-19 10:14:05.0	Interview Scheduled												

Diagram 3.7 : Candidate Dashboard Interface

Diagram 3.7 show Candidate Dashboard Interface for Candidate sign in user that allow user to see their job information, application tracking and search job.

Search Jobs	
web Search	
web development Location: Puncak Alam, Selangor Salary: RM4500 fresh graduate 4 days in office, 2 days at work from home	Apply Now
game developer Location: Johor Bahru, Johor Salary: RM5000 Bachelor/BA/BS Degree or Diploma in Computer Science, Game Development, or related field Strong knowledge of programming languages such as C#, C++, or Java Experience with game engines like Unity or Unreal Engine Understanding of game physics, AI, and animation systems We are looking for a creative and passionate Game Developer to design, develop, and maintain interactive games for web and mobile platforms. You will be responsible for building game mechanics, optimizing performance, and collaborating with designers and artists to deliver engaging user experiences.	Apply Now
Software Engineer Location: Cyberjaya, Malaysia Salary: RM6000 Degree in Computer Science or related field - Experience with Java, Spring Boot, JSP, or PHP - Knowledge of MySQL database design - Familiarity with	Apply Now

Diagram 3.8 : Search Bar Interface

Diagram 3.8 show Search bar Interface for candidates to make research about available job.

Job Application Form

Candidate: Adam
Job ID: 3

Education
Bachelor Degree, Diploma, Certifications etc.

Skills
Java, SQL, Spring Boot, HTML, CSS

Experience
Describe your working experience

Expertise
Software Development, Database Design, Networking

Submit Application

Diagram 3.9 : Job Application Interface

Diagram 3.9 show Job Application interface that need candidate to fulfill their information such as education info, skill, experience and expertise before applying for a job.

Application Tracking			
Job	Date	Status	
web development	2026-06-20 05:59:39.0	INTERVIEW SCHEDULED	
game developer	2026-06-19 22:40:11.0	SHORTLISTED	
architect	2026-06-19 10:14:05.0	INTERVIEW SCHEDULED	

My Interviews			
Job	Date	Time	Location
architect	2026-06-30	09:00:00	Google Meets
web development	2026-07-02	10:00:00	Google Meets

Diagram 3.10 : History Interface

Diagram 3.10 show History Interface. They can see their previous application and update for their job application and interview schedule.

4. Architecture Evaluation

This section evaluates the implemented Event-Driven Job Recruitment & Application Processing System based on the defined non-functional requirements, including scalability, availability, persistence, and responsiveness. The evaluation ensures that the system architecture aligns with the selected Event-Driven Architecture (EDA) design and supports reliable recruitment workflow processing.

4.1 Elastic Scalability

The system was designed with modular services such as User Management Service, Job Service, Application Processing Service, and Notification Service to improve scalability. Each module handles a specific responsibility and can operate independently.

Although the prototype does not implement a full message broker such as Kafka or RabbitMQ, event-based processing is simulated through service communication and the `event_log` table in the database. This approach helps reduce tight coupling between modules and allows the system to handle increasing numbers of job applications more efficiently.

If application traffic increases significantly, additional processing services can be introduced without affecting the entire system.

4.2 High Availability

High availability ensures that the recruitment portal remains accessible even when some background services experience delays or temporary failures.

In this system, core user actions such as job searching, job application submission, and status checking remain functional independently from background processing tasks such as notification delivery or application evaluation. This separation reduces service dependency and improves system reliability.

For example, candidates can still submit applications even if notification processing is temporarily delayed.

4.3 Bulletproof Persistence

Data persistence is achieved using MySQL as the primary relational database. Important recruitment data such as user information, job postings, submitted applications, and event records are stored in dedicated tables including users, jobs, applications, and event_log.

The use of foreign key constraints ensures referential integrity between related tables. For example, every application record must reference a valid job and candidate record, reducing the risk of invalid data storage.

Additionally, the event log provides traceability for system events, ensuring important processing steps can be monitored and audited.

4.4 Responsiveness

System responsiveness was evaluated by observing response time during major user interactions such as login, searching jobs, and applying for jobs.

Heavy backend processing such as application evaluation is separated from direct user interaction, allowing the system to provide faster responses on the frontend. This reduces waiting time for users and improves overall experience.

The implemented prototype achieved acceptable response time for normal workload, with most user actions completing within the targeted 2-second response threshold.

5. Team Contributions

Name	Role	Contribution
Muhammad Radilan bin Yaakub	Backend & Deployment Lead	JSP frontend development including login page, dashboard, job application form, and application tracking page Java Servlet backend development including authentication, job search, and application processing modules MySQL database schema design, table creation, and SQL query implementation. Event processing service implementation and debugging Apache Tomcat deployment, server configuration, and bug fixing.
Muhammad Adam Danish bin Mohamad Faiz	Full Stack & Architecture Lead	Event-Driven Architecture (EDA) system design and planning Architecture diagrams development (System Context, Component, Interaction, Deployment Diagram), system module integration and event flow planning. Project documentation, architecture evaluation, and final report preparation, System testing and performance evaluation.

6. Conclusion and Reflection

6.1 Conclusion

The Event-Driven Job Recruitment & Application Processing System was successfully developed to address common problems in traditional recruitment processes such as slow application processing, inefficient communication between recruiters and candidates, and difficulty managing large numbers of job applications. By applying Event-Driven Architecture (EDA), the system is able to process recruitment activities more efficiently through event-based communication between different modules.

The implemented system consists of several main modules including User Management, Job Management, Application Processing, Interview Scheduling, and Notification services. These modules work together to handle important recruitment workflows such as job posting, job application submission, application evaluation, and status tracking. The use of event-driven processing helps reduce dependency between modules and improves system flexibility.

Throughout this project, system requirements were analysed, architecture diagrams were designed, suitable technologies were selected, and a working prototype was successfully implemented using Java Servlet, JSP, Apache Tomcat, and MySQL. Based on the architecture evaluation, the system demonstrates good responsiveness, maintainability, and scalability for handling recruitment-related processes. Therefore, this architecture provides a strong foundation for developing a modern and efficient recruitment platform.

6.2 Reflection

This project gave us valuable experience in understanding software architecture and how architectural design affects system performance and maintainability. Through this project, we learned how to analyse system requirements, design architecture diagrams, implement system components, and evaluate whether the selected architecture fits the project objectives.

One of the most important lessons learned was understanding how Event-Driven Architecture improves loose coupling between modules and supports asynchronous processing. This helped us understand how different services can communicate without depending directly on each other, making the system more scalable and easier to maintain.

Besides technical skills, this project also improved our teamwork, communication, and problem-solving abilities. During development, we faced several challenges such as database integration, servlet debugging, event flow design, and resolving application submission errors. Solving these issues helped us gain better practical knowledge and strengthened our confidence in system development.

Overall, this project successfully improved our understanding of software architecture principles and provided useful hands-on experience that will benefit us in future software engineering projects.

References

1. *What is event-driven architecture? A complete guide.* (2025, September 11). Technology & Software Development Blog | Future Processing.
<https://www.future-processing.com/blog/event-driven-architecture-guide/>
2. GeeksforGeeks. (2024, January 31). *EventDriven Architecture System Design.* GeeksforGeeks.
<https://www.geeksforgeeks.org/system-design/event-driven-architecture-system-design>
3. Richardson, C. (2017). *Microservices.io.* Microservices.io; Chris Richardson.
<https://microservices.io/patterns/data/event-driven-architecture.html>
4. Navarro, J., & Parrend, P. (2018). *Clustering analysis classes to identify software component*, 76.
<https://www.sciencedirect.com/science/article/abs/pii/S095058491400130X>
5. sallam, mohammed. (2025, August 26). *Fundamentals of software architecture Chapter 14: Event-Driven Architecture Style.* Medium.
<https://medium.com/@mohamedsallam953/fundamentals-of-software-architecture-chapter-14-event-driven-architecture-style-ffea4ecbbce1>